

Session 9 : TP — Moteur de Particules (VFX)

Objectifs du TP

- Construire un **moteur de particules** : émettre, mettre à jour, recycler.
- Modéliser les **forces** sur une particule : gravité, drag, flottabilité, turbulence.
- Intégrer avec **Euler** — et comprendre **pourquoi** c'est légitime ici (vs Verlet en session 8).
- Gérer la **collision** avec le sol : impulsion, restitution, perte d'énergie.
- Utiliser un **object pool** pour éviter les allocations par frame.
- Produire trois effets avec le **même** moteur : pluie, feu, fumée.

💡 Contexte

Un système de particules simule un grand nombre de petits objets éphémères : pluie, feu, fumée, étincelles. Contrairement au tissu de la session 8, chaque particule vit peu de temps et est très amortie — l'intégration d'Euler suffit, et la physique (forces, collisions) est le cœur du travail. Le rendu (shader, BufferGeometry) est **fourni**.

1. La Particule et l'Object Pool

Propriétés d'une particule

- position, velocity (Vector3)
- color (Color), size, alpha (transparence)
- life (temps restant), maxLife (durée totale)
- alive (booléen — active ou recyclable)

💡 Object Pooling — le point clé

Créer/détruire des objets à chaque frame provoque des pics de **garbage collection** et des saccades. On alloue donc **une fois** un tableau fixe de MAX_PARTICLES, et on **recycle** les particules mortes en les réinitialisant plutôt qu'en les supprimant. Zéro allocation pendant le jeu.

```
const pool = [];  
for (let i = 0; i < MAX_PARTICLES; i++) {  
    pool.push({ position: new THREE.Vector3(), velocity: new THREE.Vector3(),  
                color: new THREE.Color(), alpha: 0, size: 1,  
                life: 0, maxLife: 1, alive: false });  
}
```

2. L'Émetteur

Emitter

Génère des particules à un **taux** donné (ex: 300/s) à partir d'une position emitterOrigin. La **forme** de l'émetteur contrôle la position et la direction initiales :

- **Point** : tout part du même endroit, direction fixe.
- **Cône** : direction déviée aléatoirement dans un cône d'ouverture spread.
- **Sphère** : position aléatoire sur une sphère, direction = normale sortante.

💡 Taux précis avec un accumulateur

On ne peut pas émettre un nombre fractionnel de particules par frame. On accumule le reste : `spawnAccumulator += rate * dt`, on émet `floor(spawnAccumulator)`, et on soustrait ce qu'on a émis. Le taux reste exact en moyenne.

3. Le Cycle de Vie

Cycle d'une particule

1. **Naissance** — `spawnParticle(p)` : position, vitesse, `life = maxLife`, couleur de départ.
2. **Update** — `updateParticle(p, dt)` : forces (gravité, drag), intégration d'Euler, `life -= dt`.
3. **Rendu** — `updateBuffers()` : recopie l'état dans la `BufferGeometry`.
4. **Mort / Recyclage** — quand `life <= 0`, `alive = false` puis `recycleParticle(p)` relance la particule à l'émetteur.

💡 Euler, pas Verlet — et pourquoi ici c'est légitime

En session 8, on a **banni** Euler au profit de Verlet pour le tissu : Euler dérive en énergie, et un drapeau qui oscille pendant des heures finit par exploser. Pour la VFX, on **revient** à Euler — délibérément. Trois raisons :

1. **Durée de vie courte.** Une particule de feu vit ~ 1 s, une goutte de pluie ~ 1.5 s. La dérive d'Euler est proportionnelle au temps d'intégration — sur 1 s, elle est **invisible**, quand sur 10 min elle faisait exploser le tissu.
2. **Amortissement fort.** Le drag dissipe l'énergie à chaque frame. Même si Euler en ajoute un peu, le drag en retire plus — le système est **globalement dissipatif**, pas conservatif. C'est l'inverse du tissu, où l'énergie devait être conservée.
3. **Pas de contraintes couplées.** Verlet brillait pour PBD (corriger des positions satisfait les contraintes automatiquement). Une particule VFX est **libre** — aucune contrainte de distance à maintenir. Euler suffit.

Quand même utiliser Verlet pour des particules ? Si on relie les particules entre elles (corde, tissu, ragdoll) — mais alors on retombe sur le cas de la session 8.

Le pas de temps en VFX

On utilise souvent un `dt` **fixe** (ex: $\frac{1}{60}$ s) plutôt que le vrai `dt` variable. Pourquoi ? Un `dt` variable peut donner des comportements différents selon le FPS (une étincelle qui va plus loin à 30 FPS qu'à 60 FPS). Avec un `dt` fixe, la simulation est **reproductible** — important pour déboguer un effet. Le prix : une légère désynchronisation physique/rendu, négligeable pour la VFX.

4. Forces sur une Particule

Le cœur physique du moteur est dans `updateParticle` : à chaque frame, on calcule l'accélération totale, on intègre. La qualité de l'effet dépend entièrement des **forces** qu'on choisit.

4.A. Gravité

Gravité constante

La force la plus simple : $\vec{F}_g = m \cdot \vec{g}$, avec $\vec{g} = (0, -9.81, 0)$ “m/s²”. L’accélération est $\vec{a} = \vec{g}$ (indépendante de la masse – Galilée). Pour la pluie, c’est l’essentiel.

4.B. Drag – linéaire vs quadratique

Drag linéaire (Stokes)

Force opposée à la vitesse, **proportionnelle** à v :

$$\vec{F}_{\text{drag}} = -k \cdot \vec{v}$$

- k : coefficient de frottement (kg/s).
- Valable pour des **petits** objets **lents** dans un fluide visqueux (goutte de brouillard, particule de fumée).
- La vitesse **terminale** est $v_\infty = \frac{g}{k}$: la particule cesse d’accélérer.

Drag quadratique (turbulent)

Pour des objets plus gros ou plus rapides, la traînée dépend du carré de la vitesse :

$$\vec{F}_{\text{drag}} = -\frac{1}{2}\rho C_d A \cdot \|\vec{v}\| \cdot \vec{v}$$

- ρ : densité du fluide, C_d : coefficient de traînée, A : section.
- On simplifie en $\vec{F}_{\text{drag}} = -k \cdot \|\vec{v}\| \cdot \vec{v}$.
- Valable pour une **goutte de pluie réelle** ($v \sim 9$ m/s) ou un projectile.
- La vitesse terminale est $v_\infty = \sqrt{\frac{2mg}{\rho C_d A}}$.

💡 Lequel choisir ?

Dans le TP, on utilise le drag **linéaire** (plus simple, stable avec Euler). Pour la pluie réaliste, le drag quadratique donne une vitesse terminale plus nette. La différence visuelle : avec le drag linéaire, la particule ralentit **graduellement** ; avec le quadratique, elle **plafonne** brusquement à v_∞ .

Implémentation des deux drags.

```
// Drag linéaire (utilisé dans le TP)
const dragLin = velocity.clone().multiplyScalar(-k);

// Drag quadratique (bonus)
const speed = velocity.length();
const dragQuad = velocity.clone().multiplyScalar(-k * speed);

// L'accélération totale :
// a = g + F_drag / m
```

4.C. Flottabilité — pourquoi le feu monte

Poussée d'Archimède

Un volume V de fluide (air chaud, fumée) subit une force **vers le haut** égale au poids de l'air déplacé :

$$\vec{F}_{\text{buoy}} = -\rho_{\text{air}} \cdot V \cdot \vec{g}$$

(le signe $-$ car $\vec{g}$ pointe vers le bas, la poussée est vers le haut).

Le poids de la particule elle-même est $\vec{F}_g = m \cdot \vec{g}$. La force **nette** verticale est :

$$\vec{F}_{\text{net}} = (m - \rho_{\text{air}} V) \cdot \vec{g} = m \cdot \vec{g} \cdot \left(1 - \frac{\rho_{\text{air}}}{\rho_{\text{particule}}}\right)$$

puisque $\rho_{\text{particule}} = \frac{m}{V}$.

Le raccourci du TP

Plutôt que de modéliser densité et volume, on **fusionne** la gravité et la flottabilité en une seule « gravité effective » :

$$g_{\text{eff}} = g \cdot \left(1 - \frac{\rho_{\text{air}}}{\rho_{\text{particule}}}\right)$$

- Si $\rho_{\text{particule}} > \rho_{\text{air}}$ (pluie, étincelle) : $g_{\text{eff}} < 0$ — ça tombe.
- Si $\rho_{\text{particule}} < \rho_{\text{air}}$ (feu, fumée) : $g_{\text{eff}} > 0$ — ça monte.

C'est exactement ce que fait le preset fire avec gravity: +2.5 : une gravité **positive** = flottabilité nette vers le haut. Pas un hack — une simplification légitime d'Archimède.

4.D. Turbulence — vent et flicker

Force de turbulence

Pour que la fumée ondule et le feu crépite, on ajoute une force **pseudo-aléatoire** qui varie dans le temps et l'espace. Une approche simple et peu coûteuse : des sinusoides déphasées (comme le vent du drapeau en session 8) :

$$\vec{F}_{\text{turb}}(\vec{r}, t) = A \cdot (\sin(\omega_1 t + k_1 x), \sin(\omega_2 t + k_2 y), \sin(\omega_3 t + k_3 z))$$

- A : amplitude, ω_i : pulsations, k_i : nombres d'onde (variations spatiales).
- **Pas** du vrai bruit de Perlin — mais visuellement convaincant et $O(1)$ par particule.

Turbulence dans updateParticle.

```
const t = performance.now() * 0.001;
const turbX = TURBULENCE * Math.sin(t * 3.0 + p.position.y * 1.5);
const turbZ = TURBULENCE * Math.sin(t * 4.0 + p.position.x * 1.2);
// Ajouter à l'accélération (en plus de la gravité et du drag)
ax += turbX;
az += turbZ;
```

Régler TURBULENCE à 0 pour de la pluie (trajectoires droites), à ~ 3 pour de la fumée (ondulations), à ~ 5 pour un feu qui crépite.

5. Collision avec le Sol

Détection et réponse

Quand une particule traverse le sol ($y < y_{\text{sol}}$) :

1. **Détection** : `if (p.position.y < GROUND_Y)`
2. **Correction de position** : `p.position.y = GROUND_Y` (on remonte à la surface).
3. **Réponse en impulsion** : on inverse la composante normale de la vitesse, atténuée par un coefficient de **restitution** $e \in [0, 1]$:

$$v_{y'} = -e \cdot v_y$$

- $e = 1$: rebond parfait (énergie conservée) — irréaliste.
- $e = 0$: la particule s'écrase (perte totale d'énergie normale).
- $e \sim 0.3$: rebond mou, typique d'une goutte ou d'une étincelle.

💡 Pourquoi seulement la composante y ?

Le sol est un plan **horizontal**. La normale du plan est $(0, 1, 0)$ — seule la composante v_y est affectée par le rebond. Les composantes v_x, v_z (tangentielles) sont **conservées** (en l'absence de frottement de surface). C'est la décomposition **normale/tangentielle** vue en session 6, appliquée à un plan.

Collision dans `updateParticle`.

```
const GROUND_Y = -2;
const RESTITUTION = 0.3;

// ... après intégration ...
if (p.position.y < GROUND_Y) {
  p.position.y = GROUND_Y;           // correction
  p.velocity.y = -RESTITUTION * p.velocity.y; // rebond
  // Bonus : éclaboussure — accélérer la mort
  if (Math.abs(p.velocity.y) < 0.5) p.life = Math.min(p.life, 0.2);
}
```

La dernière ligne simule l'éclaboussure : une goutte qui touche le sol avec une faible vitesse « meurt » vite (elle s'étale et disparaît).

6. Rendu — Points + Shader

💡 Cette section est **fournie** — pas au programme du TP

Le rendu (BufferGeometry, shader, blending) est déjà implémenté dans le fichier de départ. Cette section explique **ce qui est en place** pour que vous compreniez le code fourni — vous n'avez **pas** à écrire le shader ni à gérer la BufferGeometry. Concentrez-vous sur la **physique** (sections 4 et 5).

`BufferGeometry` : la géométrie sur le GPU

En Three.js, une BufferGeometry est un conteneur de **tableaux de données brutes** (Float32Array) envoyés directement à la carte graphique. Pas de hiérarchie de sommets/faces :

juste des **attributs** alignés en mémoire, un par propriété. C’est le format le plus proche du GPU — et le seul performant pour des milliers de particules.

Chaque **attribut** est un tableau plat où les données d’une particule i occupent un bloc contigu :

$$\text{position}_i = (\text{array}[3i], \text{array}[3i + 1], \text{array}[3i + 2])$$

Dans notre moteur, quatre attributs décrivent chaque particule :

Attribut	Dimension	Rôle
position	3 (xyz)	Position dans le monde — utilisée par le vertex shader pour placer le point.
color	3 (rgb)	Couleur — interpolée sur la vie (jaune → rouge pour le feu).
aSize	1	Taille en pixels du point sprite — atténuée par la distance dans le shader.
aAlpha	1	Transparence $\alpha \in [0, 1]$ — 0 = invisible (morte ou fondu final).

Table 1: Quatre attributs par particule, stockés dans une BufferGeometry et lus par le shader.

💡 Pourquoi `BufferGeometry` plutôt que des `Mesh` ?

Dessiner 3000 Mesh séparés = 3000 **draw calls** (un par sphère) — le CPU devient le goulot d’étranglement. Avec `THREE.Points` + une BufferGeometry, les 3000 particules partagent **une seule géométrie** et **un seul matériau** : c’est **un seul draw call**. Le GPU fait tout le travail en parallèle. C’est la différence entre 60 FPS et 5 FPS.

`THREE.Points` — un point par sommet

`THREE.Points` est l’objet qui dit à Three.js : « dessine chaque sommet de la géométrie comme un **point** (un carré de pixels), pas comme un triangle ». La taille du carré est contrôlée par `gl_PointSize` dans le **vertex shader** — d’où l’attribut `aSize`. Le **fragment shader** décide ensuite de la couleur et de la forme finale (disque doux via `discard` + `smoothstep`).

💡 Comment le rasterizer sait-il quoi afficher ?

Pour un triangle, le rasterizer remplit l’intérieur des 3 sommets projetés. Mais un point n’a qu’un sommet — comment obtient-on des pixels ?

Le GPU a un mode primitif dédié (`GL_POINTS`) : à partir de la position projetée (`gl_Position`) et de la taille (`gl_PointSize`), il génère **automatiquement** un carré de $N \times N$ pixels centré sur le sommet. Pas de triangle, pas de géométrie supplémentaire — la taille **est** la géométrie.

Dans ce carré, chaque pixel reçoit une coordonnée `gl_PointCoord` $\in [0, 1]^2$ (centre = $(0.5, 0.5)$), **générée par le rasterizer** — pas un attribut qu’on fournit. C’est ce qui permet au fragment shader de calculer la distance au centre et de dessiner un disque :

```
vec2 uv = gl_PointCoord - 0.5;    // centré
if (length(uv) > 0.5) discard;    // carré -> disque
```

Comment le pipeline sait-il qu'il doit utiliser ce mode ? Ce n'est pas le shader qui décide — c'est le **type primitif du draw call**. Three.js choisit `gl.POINTS` vs `gl.TRIANGLES` selon la classe de l'objet (`THREE.Points` → points, `THREE.Mesh` → triangles). C'est cette constante, passée à `gl.drawArrays(...)`, qui active le mode point du rasterizer. Le **même** vertex shader pourrait tourner dans les deux modes — seul le draw call change.

4.A. Le Vertex Shader — Mesh vs Points

Jusqu'ici, toutes les sessions utilisaient `MeshStandardMaterial` : Three.js générait le shader pour nous. Pour les particules, on écrit le nôtre — et il est **très différent**. Comparons.

Le shader « habituel » — `'MeshStandardMaterial'` (simplifié)

Voici l'essentiel de ce que Three.js génère pour un Mesh. On ne l'écrit jamais à la main, mais c'est ce qui tourne sous le capot depuis la session 1.

```
// Vertex shader (Mesh) — un sommet parmi les 3 d'un triangle
attribute vec3 position;    // lieu du sommet dans le modèle
attribute vec3 normal;      // normale pour l'éclairage
attribute vec2 uv;          // coordonnée de texture

uniform mat4 modelViewMatrix; // modèle -> vue (caméra)
uniform mat4 projectionMatrix; // vue -> écran (perspective)
uniform mat3 normalMatrix;    // transpose-inverse pour les normales

varying vec2 vUv;            // UV interpolée vers le fragment
varying vec3 vNormal;        // normale interpolée vers le fragment

void main() {
    vUv = uv;
    vNormal = normalize(normalMatrix * normal);
    gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    // Pas de gl_PointSize : la taille du triangle est déduite des 3 sommets
    // projetés. Le rasterizer remplit l'intérieur du triangle.
}
```

Trois choses à remarquer :

1. `normal` et `normalMatrix` servent à l'**éclairage** (diffuse, spéculaire) — calculé dans le fragment shader à partir de `vNormal`.
2. `uv` est interpolée à **travers le triangle** (varying) pour qu'on puisse sampler une texture par pixel.
3. **Aucune** `gl_PointSize` : la taille du triangle émerge de la projection des 3 sommets, le rasterizer remplit l'intérieur.

Notre shader Points — une particule = un sommet = un carré

Voici le vertex shader que vous trouverez dans `session09_particles.js`. Beaucoup plus court, et conceptuellement différent.

```
attribute vec3 position;    // position de la particule dans le monde
attribute vec3 color;        // couleur (vertexColors: true)
```

```

attribute float aSize;          // taille en pixels du sprite
attribute float aAlpha;         // transparence

uniform mat4 modelViewMatrix;
uniform mat4 projectionMatrix;
uniform float uPixelRatio;     // retina / non-retina

varying vec3 vColor;
varying float vAlpha;

void main() {
    vColor = color;
    vAlpha = aAlpha;
    vec4 mv = modelViewMatrix * vec4(position, 1.0);
    gl_PointSize = aSize * uPixelRatio * (100.0 / -mv.z); // <-- la clé
    gl_Position = projectionMatrix * mv;
}

```

Ce qui change par rapport au Mesh :

1. **Pas de normal, pas d'uv, pas d'éclairage.** Une particule est un billboard plat face caméra — pas de notion de « surface qui reçoit la lumière ». La couleur vient directement de l'attribut `color`.
2. **gl_PointSize est obligatoire.** C'est lui qui décide de la taille du carré de pixels dessiné pour ce sommet. Sans cette ligne, rien ne s'affiche (taille = 0 par défaut).
3. **L'atténuation par la distance** ($100.0 / -mv.z$) : $mv.z$ est la profondeur dans l'espace caméra (négative devant la caméra). Plus la particule est loin, plus $|-mv.z|$ est grand, plus `gl_PointSize` est petit — comme un objet qui rapetisse avec la distance.
4. **Pas d'interpolation de varying à travers une surface.** `vColor` et `vAlpha` sont fixés **une fois** par le vertex shader ; seul `gl_PointCoord` (généré automatiquement par pixel du sprite) varie dans le fragment shader.

💡 Mesh vs Points en une ligne

Un Mesh projette **3 sommets** et remplit le **triangle** qu'ils forment (avec UVs, normales, éclairage).
 Un Point projette **1 sommet** et dessine un **carré de pixels** centré dessus, dont la taille est explicitement `gl_PointSize`. C'est pourquoi 3000 particules = 3000 invocations du vertex shader + 1 draw call, contre 3000 sphères = 300 000 sommets + 3000 draw calls.

Le pipeline en une frame

1. `updateBuffers()` recopie l'état du pool (CPU) vers les `Float32Array` des attributs.
2. On flaggue `attribut.needsUpdate = true` pour signaler à Three.js qu'il faut re-uploader les données — sinon le GPU garde l'ancienne version.
3. `renderer.render()` lance **un** draw call : le vertex shader tourne pour chaque particule, le fragment shader pour chaque pixel des points.

💡 Pourquoi `needsUpdate` ? CPU et GPU ont des mémoires séparées

Le `Float32Array` qu'on modifie en JS vit dans la mémoire **CPU**. Le GPU, lui, garde sa propre copie des attributs dans sa **VRAM**. Quand on mute le tableau JS, le GPU ne « voit » rien — il faut explicitement **uploader** les nouvelles données.

`needsUpdate` est le **flag Three.js** qui déclenche cet upload : au prochain `render()`, Three.js recopie le tableau vers le GPU puis remet le flag à `false`. C'est une commodité Three.js — en WebGL brut, on appellerait `gl.bufferSubData(...)` à la main. La **contrainte** (mémoires séparées) est matérielle ; le **flag** est un détail d'API.

Trois idées de rendu

- **THREE.Points** + `BufferGeometry` : un seul draw call pour des milliers de particules. Attributs par particule : `position`, `color`, `aSize`, `aAlpha`.
- **Disque doux** : le fragment shader écarte (`discard`) ce qui dépasse un cercle et adoucit le bord avec `smoothstep` — sinon on a des carrés moches.
- **Couleur / taille / alpha sur la vie** : avec $t = \frac{\text{life}}{\text{maxLife}}$, on interpole la couleur (départ → fin), l'alpha (fendu) et la taille. C'est ce qui fait qu'un feu passe du jaune au rouge sombre en s'estompant.

💡 Additive blending pour le feu

`material.blending = THREE.AdditiveBlending` additionne les couleurs des particules superposées : les flammes se cumulent et **brillent**. Indispensable pour le feu, à éviter pour la pluie/fumée (qui doivent rester opaques).

💡 Et avec WebGPU ? (pour culture)

Notre TP utilise WebGL, où la physique tourne sur le **CPU** (en JS) et les données sont **uploadées** vers le GPU à chaque frame (`needsUpdate`). C'est suffisant pour quelques milliers de particules.

WebGPU, l'API moderne qui remplace WebGL, change deux choses :

1. **Compute shaders** : la physique (gravité, drag, collisions) peut tourner **entièrement sur le GPU** dans un programme générique. Les données ne quittent jamais la VRAM — le CPU ne fait rien par frame. C'est ainsi qu'on simule des **millions** de particules.
2. **Fin de GL_POINTS** : WebGPU n'a plus de primitive « point » intégrée. On dessine les particules avec du **instancing** — un quad (2 triangles) répliqué par particule, positionné et dimensionné dans le vertex shader. Plus de `gl_PointSize` ni `gl_PointCoord` — on utilise de vraies UVs.

La **physique** que vous apprenez ici (forces, intégration, collisions) reste identique — seul l'endroit où elle s'exécute change. Le dossier `three.js/examples/` du projet contient des démos WebGPU de particules (`webgpu_compute_particles_snow.html`, `_rain.html`, `_fluid.html`) si vous voulez voir la différence.

7. TP — Moteur de Particules

💡 Objectif du TP

Compléter le fichier `session09_particles.html` (via Live Server). La scène, le pool, la `BufferGeometry`, le shader, la GUI et la boucle sont fournis — il reste **cinq fonctions** à écrire. Une fois en place, le sélecteur de **preset** (pluie / feu / fumée) donne trois effets très différents avec le même moteur.

Mise en place

- Ouvrir `session09_particles.html` dans le navigateur.
- Au départ, rien ne s'affiche : les 5 fonctions sont vides, aucune particule n'est initialisée. C'est normal.
- La GUI propose : taux d'émission, gravité, drag, lifetime, forme de l'émetteur, spread, preset, reset, FPS.

Missions

Mission 1 — `spawnParticle(p)`

Récupérer { `pos`, `dir` } via `sampleEmitter`, placer la particule, donner une vitesse initiale (`ps.speed` avec un peu d'aléa), régler `life = maxLife = LIFETIME` (avec un peu d'aléa), la couleur de départ du preset, `alpha = 1`, `size = ps.size`, et `alive = true`.

Mission 2 — `updateParticle(p, dt)`

Appliquer gravité (section 4.A) + drag linéaire (4.B), intégrer en Euler, décrémenter `life`. Si `life <= 0`, `alive = false`. Calculer $t = \frac{\text{life}}{\text{maxLife}}$ et interpoler couleur / alpha / taille sur la vie. **Bonus** : ajouter la turbulence (4.D) et la collision avec le sol (section 5).

Mission 3 — `recycleParticle(p)`

Relancer la particule morte en appelant `spawnParticle(p)` — elle retourne à l'émetteur.

Mission 4 — `updateBuffers()`

Recopier `position`, `color`, `aSize`, `aAlpha` du pool vers les attributs de la `BufferGeometry`. Une particule morte a `aSize = aAlpha = 0` (invisible). Flagger `needsUpdate = true`.

Mission 5 — `sampleEmitter(shape, spread)`

Renvoyer { `pos`, `dir` } selon la forme. Le preset fournit `emitterPos` et `dir` de base. **Point** : `pos = emitterPos`, `dir = dir` du preset. **Cône** : `dir = dir` du preset + déviation aléatoire dans un cône d'ouverture `spread`. **Sphère** : `pos = point` aléatoire sur une sphère de rayon `spread` autour de `emitterPos`, `dir = normale` sortante. Indices : `Vector3.randomDirection()` pour la sphère ; `Quaternion().setFromUnitVectors(+Z, dir)` pour orienter le cône.

Scénarios à observer

- **Pluie** : gravité négative forte (section 4.A), blending normal, forme point, émetteur en hauteur. Les gouttes tombent en ligne droite et disparaissent au sol. Avec la collision (section 5), elles rebondissent faiblement.
- **Feu** : gravité **positive** = flottabilité (section 4.C), **additive blending**, forme cône, émetteur près du sol. Les particules montent, passent du jaune au rouge sombre et s'estompent. Avec la turbulence (4.D), le feu crépite.

- **Fumée** : gravité positive légère (flottabilité faible), drag élevé (section 4.B), blending normal, forme cône large. Montée lente, gris qui fonce, longue durée de vie. La turbulence la fait onduler.
- **Euler vs Verlet** : observer qu’aucune explosion ne se produit même après des minutes — preuve que le drag dissipe l’énergie qu’Euler pourrait ajouter (section 3).
- **Performance** : avec 3000 particules et un pool, le FPS reste stable — preuve que l’object pooling évite les pics de GC.

⚠ Pièges à éviter

- **Ne pas allouer** de Vector3/Color dans updateParticle ou updateBuffers — c’est justement ce que le pool évite. Réutiliser les champs existants (`p.position.add(...)`, `p.color.lerpColors(...)`).
- Oublier `needsUpdate = true` sur les attributs modifiés → l’écran ne bouge pas.
- Une particule morte doit avoir `aAlpha = 0` (et idéalement `aSize = 0`), sinon elle reste visible à l’origine.
- Tester `life <= 0` **après** intégration, et bien passer `alive = false` pour déclencher le recyclage.
- Le discard dans le shader est essentiel : sans lui, `THREE.Points` dessine des carrés.

Bonus

- **Drag quadratique** (section 4.B) : remplacer le drag linéaire par $-k \cdot \|v\| \cdot v$ et observer la vitesse terminale plus nette de la pluie.
- **Turbulence** (section 4.D) : ajouter un slider TURBULENCE dans la GUI et l’appliquer dans updateParticle. Régler à 0 pour la pluie, 3 pour la fumée, 5 pour le feu.
- **Collision avec le sol** (section 5) : ajouter le rebond avec restitution dans updateParticle, plus l’éclaboussure (mort accélérée).
- **Preset étincelles** : gravité forte, lifetime très court (~ 0.5 s), additive blending, couleur blanc → jaune → rouge, turbulence élevée.
- **Attraction** : une force vers un point mobile (la souris) — $\vec{F} = k \cdot (\text{cible} - \text{position})$. Utile pour des étincelles orbitantes ou un trou noir.